



INTERNAL

Platform Event Bus Guide

Quenyx vOPS HUB — v1.0.0 · Document v1.0

Document Version	1.0
Software Version	v1.0.0
Applies To	Quenyx vOPS HUB v1.0.0 — Sprint 25
Classification	Internal
Owner	Platform Engineering
Status	Released
Document Type	Platform guide

Table of Contents

- | [What it is](#)
- | [Components](#)
- | [Event vocabulary](#)
- | [Publishing](#)
- | [Subscribing](#)
- | [Introspection](#)
- | [Guarantees](#)

Platform Event Bus Guide

What it is

Sprint 25 introduces the **Platform Event Bus** — a shared publish/subscribe layer for platform domain events. It removes direct module-to-module calls: a publisher announces *what happened*, and any number of subscribers react, without the publisher knowing who is listening. The bus is **workspace-aware** (every event carries a `Project`), **auditable** (every publish is written to `audit_logs`), and **async-ready** (the fan-out seam is isolated so a queue worker can replace synchronous dispatch without changing a single publisher or subscriber).

Components

Piece	Path	Role
<code>PlatformEventNames</code>	<code>app/Services/Platform/EventBus/PlatformEventNames.php</code>	Canonical event vocabulary (21 events). Unknown names are rejected.
<code>PlatformEvent</code>	<code>app/Services/Platform/EventBus/PlatformEvent.php</code>	Immutable event object with fields: <code>uuid</code> , <code>name</code> , <code>workspace</code> , <code>actor</code> , <code>payload</code> , <code>occurred_at</code> , <code>correlation_id</code> .
<code>EventSubscriber</code>	<code>app/Contracts/Platform/EventSubscriber.php</code>	Subscriber contract with methods: <code>key()</code> , <code>subscribe()</code> , <code>handle()</code> .
<code>PlatformEventBus</code>	<code>app/Services/Platform/EventBus/PlatformEventBus.php</code>	Singleton: <code>publish()</code> , <code>subscribe()</code> , <code>describe()</code> , <code>recent()</code> .
<code>NotificationFanoutSubscriber</code>	<code>app/Services/Platform/EventBus/Subscribers/</code>	Example reaction to urgent events — <code>QynNotify</code> (no publisher coupling).

Event vocabulary

```
AlertCreated AlertResolved AssetCreated AssetUpdated
WorkflowStarted WorkflowCompleted WorkflowFailed
IncidentOpened IncidentUpdated IncidentResolved
TicketCreated TicketUpdated KnowledgeCreated KnowledgeUpdated
ConversationCompleted RecommendationAccepted RecommendationRejected
ApprovalGranted ApprovalRejected NotificationSent ComplianceAssessmentCompleted
```

Publishing

```
app(PlatformEventBus::class)→publish(
  PlatformEventNames::INCIDENT_OPENED,
  $project,           // workspace-aware
  $user,              // optional actor
  ['title' ⇒ $incident→title, 'severity' ⇒ $incident→severity, 'source' ⇒ 'qynreact'],
  $correlationId,     // optional, for grouping related events
);
```

- The event name **must** be one of `PlatformEventNames::all()` — typos throw `InvalidArgumentException`.
- The publish is audited (`platform_event_published`) before fan-out.
- A failing subscriber is logged and isolated; it **never** breaks publishing.

Subscribing

Implement `EventSubscriber` and register one line in `AppServiceProvider::boot()` :

```
$bus = $this→app→make(PlatformEventBus::class);
$bus→subscribe($this→app→make(MyReactionSubscriber::class));
```

`subscribedTo()` returns the event names you want (or `['*']` for all). The publisher is untouched — this is how cross-module reactions happen with **no module branching**.

Introspection

`GET /api/qynva/events` (privileged) returns the event vocabulary, the registered subscribers and which events each listens to, plus the recent in-process event ring. This also feeds **Platform Health**.

Guarantees

- **No direct module-to-module calls** — everything routes through the bus.
- **Workspace isolation** — events carry a `Project` ; subscribers re-resolve and scope by it.
- **Auditable** — every publish writes to the shared audit trail.

- **Async-ready** — synchronous + defensive today; swap the `dispatch()` body for a queue with zero changes to publishers or subscribers.